

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»**

Институт математики и информационных систем

Факультет автоматики и вычислительной техники

Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №3
по дисциплине

«Объектно-ориентированное
программирование»

Выполнил студент гр. ИВТб-2302-05-00

_____/Матешко В.В./
(Подпись)

Проверил старший преподаватель

_____/Шмакова Н.А./
(Подпись)

Киров
2026

Цель работы

Разработать веб-приложение, используя классы и особенности Java, демонстрирующее применение ООП и работу через Hibernate.

Задание

На основе предметной области разработать приложение на Java с веб-интерфейсом. Данные об объектах предметной области хранить в PostgreSQL. Для работы с БД использовать Hibernate, а для базовых CRUD-операций использовать `CrudRepository`.

Требования к реализации

1. Использование языка программирования Java;
2. Использование СУБД PostgreSQL;
3. Использование сборщика Maven;
4. Использование Hibernate для подключения к БД;
5. **Требование к БД:** не менее 3-х связанных таблиц в 3-ей нормальной форме. Обязательно хотя бы одна связь не по наследованию;
6. Веб-интерфейс.

Ход работы

Предметная область: универсальные API-подключения (VK и Telegram).

Описание классов предметной области

1. Базовый абстрактный класс Api:

- Тип класса и модификатор доступа: `public abstract class Api`;
- Поля:
 - `private Long id` — первичный ключ;
 - `protected String name` — название API-подключения;
 - `protected String baseUrl` — базовый URL;
 - `protected Organization organization` — ссылка на организацию (many-to-one).
- Методы:
 - `public String initialize()` — возвращает строку "Initializing API...";
 - `public String fetchData(String command)` — базовая реализация, возвращает строку "Загрузка данных..."; в потомках может переопределяться для предметной команды.
- Конструкторы:
 - `protected Api()` — пустой конструктор для Hibernate;
 - `protected Api(String name, String baseUrl)` — инициализирует родительские поля `name` и `baseUrl`.

2. Класс VkApi extends Api:

- Тип класса и модификатор доступа: `public class VkApi extends Api`;
- Поля:
 - `private String version` — версия VK API;
 - `private String token` — токен доступа;
 - `private List<VkPost> posts` — связанные посты (one-to-many).
- Методы:
 - `public String initialize()` — переопределённый метод инициализации, возвращает строку с полями VK (`name`, `baseUrl`, `version`, `token`);
 - `public String fetchData(String command)` — обобщённый вызов команды для демонстрации полиморфного доступа;
 - `public List<String> getText()` — возвращает список текстов постов;
 - `public List<Integer> getLike()` — возвращает список значений лайков.
- Конструкторы:
 - `protected VkApi()` — пустой конструктор для Hibernate;
 - `public VkApi(String version, String token)` — создаёт VK API с явными `version` и `token`, а `name`/`baseUrl` задаёт как "VK" и "https://api.vk.com/method";
 - `public VkApi(String token)` — создаёт VK API с токеном и версией по умолчанию "5.199".

3. Класс TgApi extends Api:

- Тип класса и модификатор доступа: `public class TgApi extends Api`;
- Поля:

- `private String botToken` — токен бота;
- `private long chatId` — идентификатор чата;
- `private List<TgPayload> payloads` — связанные данные сообщений (one-to-many).
- Методы:
 - `public String initialize()` — переопределённый метод инициализации, возвращает строку с полями TG (`name`, `baseUrl`, `botToken`, `chatId`);
 - `public String fetchData(String command)` — обобщённый вызов команды для демонстрации полиморфного доступа;
 - `public List<String> sendMessage()` — возвращает тестовые данные отправки сообщения;
 - `public List<String> sendPhoto()` — возвращает тестовые данные отправки фото.
- Конструкторы:
 - `protected TgApi()` — пустой конструктор для Hibernate;
 - `public TgApi(String botToken, long chatId)` — создаёт Telegram API с токеном и чатом, а `name`/`baseUrl` задаёт как "TG" и "https://api.telegram.org/bot";
 - `public TgApi(String botToken)` — создаёт Telegram API только с токеном, `chatId` устанавливается в NULL.

4. Класс VkPost:

- Тип класса и модификатор доступа: `public class VkPost`;
- Поля:
 - `private Long id` — идентификатор поста;
 - `private String text` — текст поста;
 - `private int likes` — количество лайков;
 - `private VkApi vkApi` — ссылка на родительский VK API (many-to-one).
- Конструкторы:
 - `protected VkPost()` — пустой конструктор для Hibernate;
 - `public VkPost(String text, int likes, VkApi vkApi)` — создаёт пост и привязывает его к VK API.

5. Класс TgPayload:

- Тип класса и модификатор доступа: `public class TgPayload`;
- Поля:
 - `private Long id` — идентификатор записи данных;
 - `private String payloadType` — тип данных (`message/photo`);
 - `private String payloadData` — содержимое данных;
 - `private TgApi tgApi` — ссылка на родительский TG API (many-to-one).
- Конструкторы:
 - `protected TgPayload()` — пустой конструктор для Hibernate;
 - `public TgPayload(String payloadType, String payloadData, TgApi tgApi)` — создаёт запись данных и привязывает её к TG API.

6. Класс Organization:

- Тип класса и модификатор доступа: `public class Organization`;

- Поля:
 - `private Long id` — идентификатор организации;
 - `private String name` — название организации;
 - `private List<Api> apis` — список связанных API-записей (one-to-many).
- Конструкторы:
 - `protected Organization()` — пустой конструктор для Hibernate;
 - `public Organization(String name)` — создаёт организацию с заданным именем.
- Методы:
 - `public void addApi(Api api)` — привязывает VK/TG API к организации (обновляет связь в обе стороны).

Модели Hibernate

Для хранения данных используются аннотации:

1. `@Entity` — помечает класс как таблицу в БД;
2. `@Table` — задаёт имя таблицы;
3. `@Id` — помечает поле как первичный ключ;
4. `@GeneratedValue(strategy = GenerationType.IDENTITY)` — автоинкремент первичного ключа средствами БД;
5. `@Column` — задаёт имя и ограничения колонки;
6. `@OneToMany` — связь «один-ко-многим» между родительской и дочерними сущностями;
7. `@ManyToOne` — связь «многие-к-одному» с хранением внешнего ключа;
8. `@JoinColumn` — задаёт имя FK-колонки.

Наследование дочерних элементов реализовано через `JOINED`. Таким образом, Hibernate создаёт отдельные таблицы для базового и дочерних классов с общим идентификатором.

Таблица `organizations`:

- `id` — первичный ключ (`BIGSERIAL, IDENTITY`);
- `name` — `VARCHAR(150) NOT NULL`.

Таблица `apis` (базовая сущность `Api`):

- `id` — первичный ключ;
- `name` — `VARCHAR(100) NOT NULL`;
- `base_url` — `VARCHAR(255) NOT NULL`;
- `organization_id` — внешний ключ на `organizations(id)`.

Таблица `vk_apis` (дочерняя к `apis`):

- `id` — первичный ключ и внешний ключ на `apis(id)`;
- `version` — `VARCHAR(50) NOT NULL`;
- `token` — `VARCHAR(255) NOT NULL`.

Таблица `tg_apis` (дочерняя к `apis`):

- `id` — первичный ключ и внешний ключ на `apis(id)`;
- `bot_token` — `VARCHAR(255) NOT NULL`;
- `chat_id` — `BIGINT NOT NULL`.

Таблица `vk_posts`:

- `id` — первичный ключ;
- `text` — `VARCHAR(1000) NOT NULL`;

- likes — INTEGER NOT NULL;
- vk_api_id — внешний ключ на vk_apis(id).

Таблица tg_payloads:

- id — первичный ключ;
- payload_type — VARCHAR(255) NOT NULL;
- payload_data — VARCHAR(1500) NOT NULL;
- tg_api_id — внешний ключ на tg_apis(id).

Связи в моделях:

Api -> Organization (many-to-one):

```
1 @ManyToOne(optional = true)
2 @JoinColumn(name = "organization_id", nullable = true)
3 protected Organization organization;
```

Organization -> Api (one-to-many):

```
1 @OneToMany(mappedBy = "organization", cascade = CascadeType.ALL, orphanRemoval = true,
   fetch = FetchType.EAGER)
2 private List<Api> apis = new ArrayList<>();
```

VkPost -> VkApi (many-to-one):

```
1 @ManyToOne(optional = false)
2 @JoinColumn(name = "vk_api_id", nullable = false)
3 private VkApi vkApi;
```

VkApi -> VkPost (one-to-many):

```
1 @OneToMany(mappedBy = "vkApi", cascade = CascadeType.ALL, orphanRemoval = true)
2 private List<VkPost> posts = new ArrayList<>();
```

TgPayload -> TgApi (many-to-one):

```
1 @ManyToOne(optional = false)
2 @JoinColumn(name = "tg_api_id", nullable = false)
3 private TgApi tgApi;
```

TgApi -> TgPayload (one-to-many):

```
1 @OneToMany(mappedBy = "tgApi", cascade = CascadeType.ALL, orphanRemoval = true)
2 private List<TgPayload> payloads = new ArrayList<>();
```

Spring подключение к PostgreSQL

Подключение и параметры Hibernate задаются в файле `application.properties`:

```
1 spring.datasource.url=jdbc:postgresql://localhost:5432/lab3_oop
2 spring.datasource.username=postgres
3 spring.datasource.password=postgres
4 spring.datasource.driver-class-name=org.postgresql.Driver
5
6 spring.jpa.hibernate.ddl-auto=update
7 spring.jpa.show-sql=true
8 spring.jpa.properties.hibernate.format_sql=true
9 spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Spring Data получает доступ к таблицам через репозитории:

```
1 public interface VkApiRepository extends JpaRepository<VkApi, Long> {}
2 public interface TgApiRepository extends JpaRepository<TgApi, Long> {}
3 public interface OrganizationRepository extends JpaRepository<Organization, Long> {}
```

Примеры запросов

Базовые JPA-операции, используемые в сервисах:

```
1 vkApiRepository.findAll();
2 vkApiRepository.findById(id);
3 vkApiRepository.save(new VkApi("5.199", token));
4 vkApiRepository.deleteById(id);
5 organizationRepository.save(org);
```

Описание API:

VK API:

- GET /api/vkapis — получить список всех VK API-записей;
- GET /api/vkapis/{id} — получить одну VK API-запись по id;
- POST /api/vkapis/full — создать VK API через конструктор (version, token);
- POST /api/vkapis/token — создать VK API через конструктор (token);
- POST /api/vkapis/default — создать VK API со значениями по умолчанию;
- PUT /api/vkapis/{id} — обновить VK API-запись по id (версия и токен);
- DELETE /api/vkapis/{id} — удалить VK API-запись по id;
- GET /api/vkapis/{id}/posts — получить список постов конкретного VK API;
- POST /api/vkapis/{id}/posts — добавить пост к конкретному VK API;
- DELETE /api/vkapis/posts/{postId} — удалить пост по id;
- GET /api/vkapis/{id}/initialize — вызвать родительский метод initialize();
- GET /api/vkapis/{id}/fetchdata/{command} — вызвать родительский метод fetchData(command);
- GET /api/vkapis/{id}/run/{command} — вызвать предметные методы VK (getText, getLike).

TG API:

- GET /api/tgapis — получить список всех TG API-записей;
- GET /api/tgapis/{id} — получить одну TG API-запись по id;
- POST /api/tgapis/full — создать TG API через конструктор (botToken, chatId);
- POST /api/tgapis/token — создать TG API через конструктор (botToken);
- POST /api/tgapis/chat — создать TG API по chatId, при этом botToken выставляется в "NULL";
- PUT /api/tgapis/{id} — обновить TG API-запись по id (botToken и chatId);
- DELETE /api/tgapis/{id} — удалить TG API-запись по id;
- GET /api/tgapis/{id}/payloads — получить список данных конкретного TG API;
- POST /api/tgapis/{id}/payloads — добавить данные к конкретному TG API;
- DELETE /api/tgapis/payloads/{payloadId} — удалить данные по id;
- GET /api/tgapis/{id}/initialize — вызвать родительский метод initialize();
- GET /api/tgapis/{id}/fetchdata/{command} — вызвать родительский метод fetchData(command);
- GET /api/tgapis/{id}/run/{command} — вызвать предметные методы TG (sendMessage, sendPhoto).

Описание графического интерфейса

Интерфейс реализован на React и включает:

- формы добавления VK и TG API записей;
- форму редактирования записей по ID (PUT для VK и TG);
- форму добавления связанных данных (VK post и TG данные);
- вывод списков связанных данных и удаление по кнопке;
- кнопки удаления записей;
- блок вызова методов предметной области (`Initialize`, `FetchData`, `run(command)`) через выпадающие списки;
- блок отображения результата вызова методов и ошибок;
- кнопку обновления списков записей из БД;
- формы добавления организации и подключения к ней API.

Ссылка на github

Github

ERD диаграмма

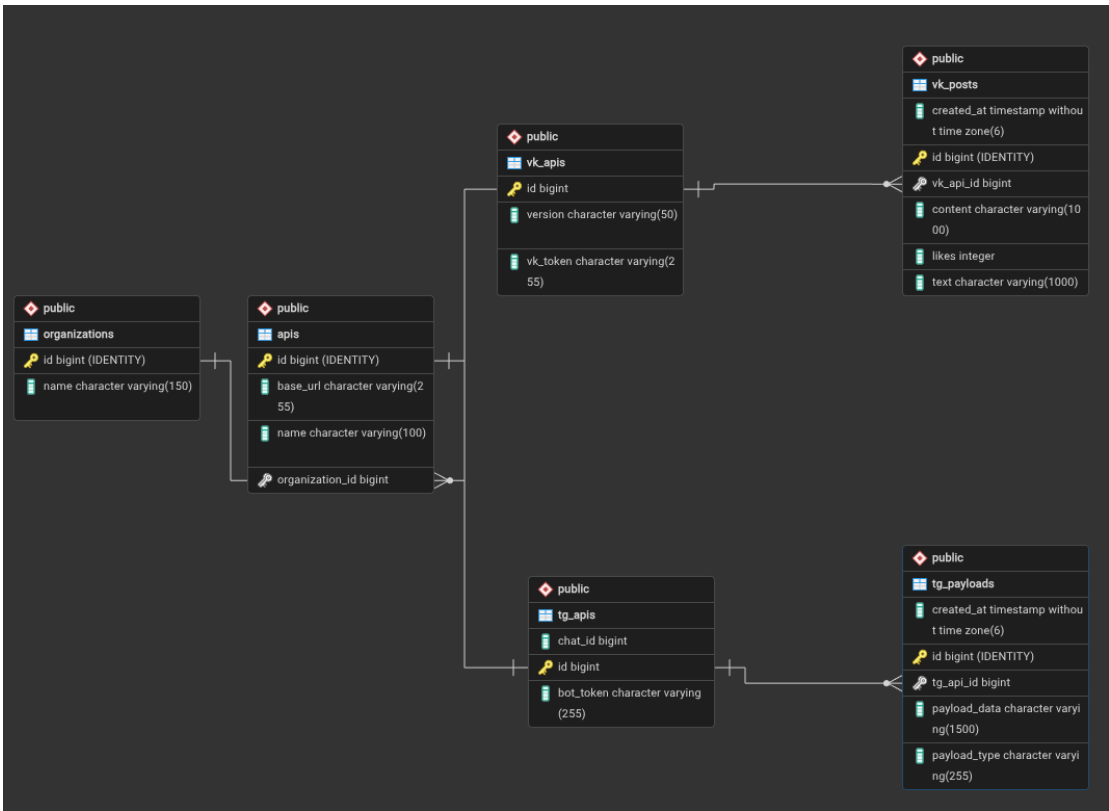


Рисунок 1 – ERD диаграмма

Диаграмма классов



Рисунок 2 – Диаграмма классов моделей

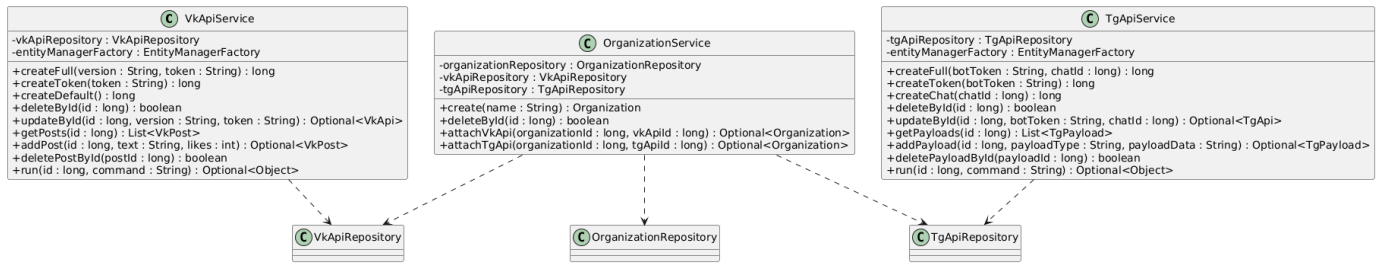


Рисунок 3 – Диаграмма классов сервисов

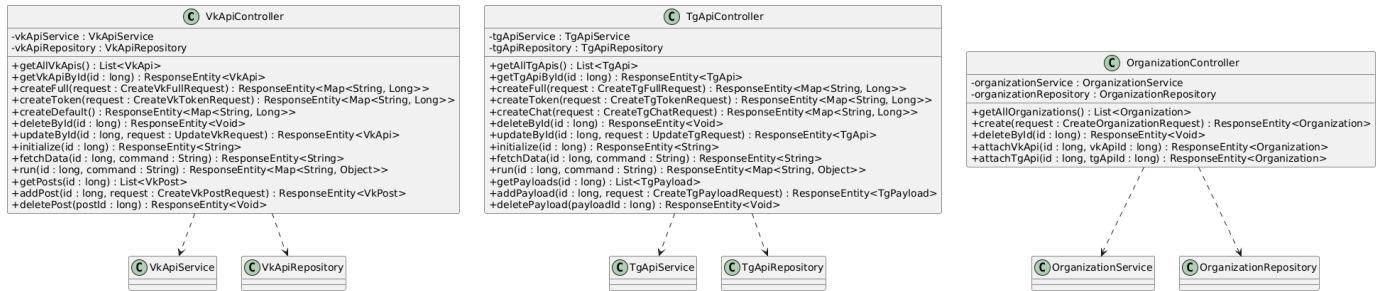


Рисунок 4 – Диаграмма классов контроллеров

Вывод

В рамках выполнения лабораторной работы было разработано web-приложение на Java для предметной области VK и Telegram API с хранением данных в PostgreSQL и работой через Hibernate. CRUD-операции реализованы через `CrudRepository`, а структура БД включает связанные таблицы и связи не только по наследованию.

В процессе выполнения работы были приобретены навыки описания модель предметной области через JPA-аннотации и работы с сервисным и контроллерным слоями Spring.